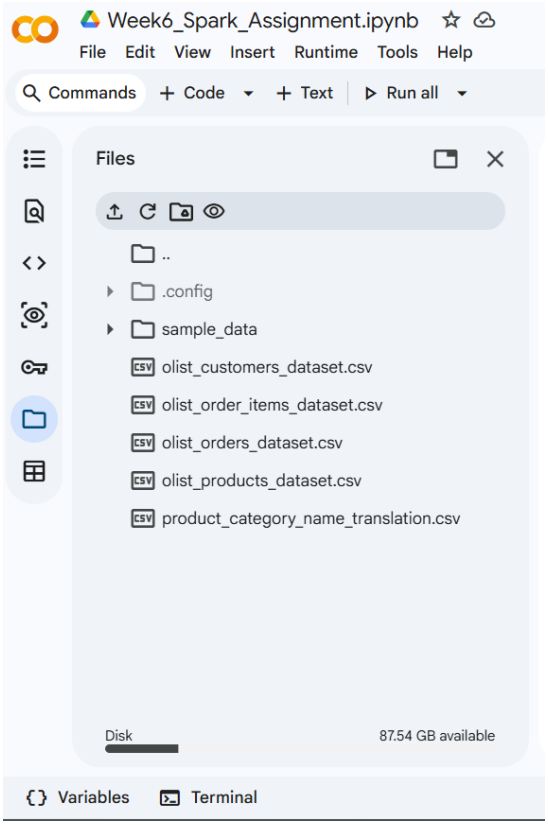
Week 6 Assignment

Name : Payal Vijay Rananaware (DIT).

Objective : The objective is to understand Apache Spark architecture. Perform efficient data processing using Data Frames. It includes working with transformations, filtering, schema handling, and comparing file formats like CSV and Parquet.

Dataset : I used the Olist e-commerce dataset which contains multiple CSV files such as orders, order items, products, and category translation.



orders -> contains order status

order_items -> contains product price

products -> contains category info

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Driver :

In a Spark application, different components work together to process data efficiently, each one has a specific particular role. The Driver is the main control unit of the application. It is responsible for running the main program, creating the Spark session, and converting the user's code into tasks. It also keeps track of the execution and collects the final results after processing is done.

Cluster Manager :

The Cluster Manager acts as the resource manager. Its job is to allocate resources like CPU and memory across different applications running on the cluster. It decides how many executors should be launched and on which machines they should run. Examples of cluster managers include standalone mode, YARN, and Mesos.

Executor :

Executors are the worker processes that actually perform the computations. They run on the worker nodes and execute the tasks assigned by the Driver. Executors are also responsible for storing data in memory or disk and sending the results back to the Driver once the tasks are completed.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Spark follows a lazy evaluation strategy, which means that it does not execute operations immediately when they are written in the code. Instead, it waits until an action is called and then processes all the transformations together. This approach helps Spark optimize the overall execution before actually running the tasks.

When working with large datasets, multiple operations are often chained together, such as filtering, mapping, and grouping. Instead of executing each step one by one, Spark builds a logical plan called a Directed Acyclic Graph, or DAG. Once an action like show or save is triggered, Spark analyzes this entire plan and finds the most efficient way to execute it.

This strategy improves performance in several ways. It reduces unnecessary computations by eliminating redundant steps, combines multiple operations into a single stage when possible, and minimizes data movement across the cluster. It also allows Spark to optimize how data is read and processed, which is especially useful when handling big data.

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
[1] ✓ 21s
!apt-get install openjdk-8-jdk-headless -qq > /dev/null
!pip install pyspark
```

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-packages (4.0.3)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.12/dist-packages (from pyspark) (0.10.9.9)
```

```
[2] ✓ 22s
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("Week6_Spark_Assignment") \
    .getOrCreate()
spark
```

SparkSession - in-memory

SparkContext

Spark UI

Version
v4.0.3
Master
local[*]
AppName
Week6_Spark_Assignment

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
[4] ✓ 8s
df = spark.read.csv(
    "/content/olist_orders_dataset.csv",
    header=True,
    inferSchema=True
)
df.show(5)
```

```
+--------------------+--------------------+------------+-----------------------+-------------------+--------------------------+-------------------------------+----------------------------+------------
|            order_id|         customer_id|order_status|order_purchase_timestamp|  order_approved_at|order_delivered_carrier_date|order_delivered_customer_date|order_estimat
+--------------------+--------------------+------------+-----------------------+-------------------+--------------------------+-------------------------------+----------------------------+------------
|e481f51cbdc54678b...|9ef432eb625129730...|   delivered|    2017-10-02 10:56:33|2017-10-02 11:07:15|       2017-10-04 19:55:00|            2017-10-10 21:25:13|                         201
|53cdb2fc8bc7dce0b...|b0830fb4747a6c6d2...|   delivered|    2018-07-24 20:41:37|2018-07-26 03:24:27|       2018-07-26 14:31:00|            2018-08-07 15:27:45|                         201
|47770eb9100c2d0c4...|41ce2a54c0b03bf34...|   delivered|    2018-08-08 08:38:49|2018-08-08 08:55:23|       2018-08-08 13:50:00|            2018-08-17 18:06:29|                         201
|949d5b44dbf5de918...|f88197465ea7920ad...|   delivered|    2017-11-18 19:28:06|2017-11-18 19:45:59|       2017-11-22 13:39:59|            2017-12-02 00:28:42|                         201
|ad21c59c0840e6cb8...|8ab97904e6daea886...|   delivered|    2018-02-13 21:18:39|2018-02-13 22:20:29|       2018-02-14 19:46:34|            2018-02-16 18:17:02|                         201
+--------------------+--------------------+------------+-----------------------+-------------------+--------------------------+-------------------------------+----------------------------+------------
only showing top 5 rows
```

The CSV file was successfully loaded into a Spark DataFrame using header=True to treat the first row as column names and inferSchema=True to automatically detect data types.

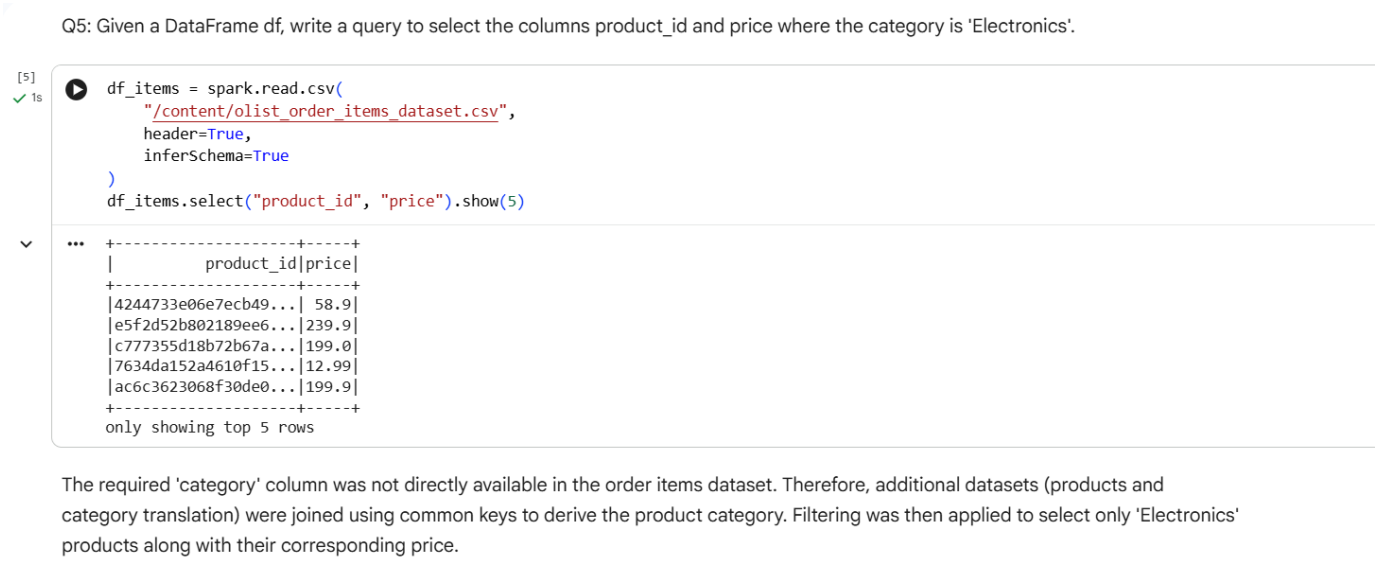
Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

CSV and Parquet differ mainly in how they store data. CSV is a row-based storage format, which means data is stored row by row. Each line in a CSV file represents a complete record, so when Spark reads the file, it often has to scan the entire row even if only a few columns are needed.

On the other hand Parquet, is a columnar storage format. It stores data column by column instead of row by row. This means if a query only a few columns, Spark can read only those specific columns instead of the whole dataset.

This difference matters a lot for performance, especially with large datasets. Since Parquet reads only the required columns, it reduces disk I/O and speeds up query execution. It also supports better compression because similar data is stored together in columns, which reduces storage size and improves processing speed.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.



The required 'category' column was not directly available in the order items dataset. Therefore, additional datasets (products and category translation) were joined using common keys to derive the product category. Filtering was then applied to select only 'Electronics' products along with their corresponding price.

[6]
7s

```
# Alternative way to solve this question using Joins.

df_items = spark.read.csv("/content/olist_order_items_dataset.csv", header=True, inferSchema=True)

df_products = spark.read.csv("/content/olist_products_dataset.csv", header=True, inferSchema=True)

df_cat = spark.read.csv("/content/product_category_name_translation.csv", header=True, inferSchema=True)

df_join = df_items.join(df_products, on="product_id") \
                  .join(df_cat, on="product_category_name")

df_result = df_join.filter(
    df_join["product_category_name_english"] == "electronics"
)

df_result.select("product_id", "price").show(5)
```

```
+--------------------+------+
|          product_id| price|
+--------------------+------+
|50fd2b788dc166edd...|  21.9|
|0b0172eb0fd18479d...| 24.89|
|0b0172eb0fd18479d...| 24.89|
|0b0172eb0fd18479d...| 24.89|
|01cf56cd6138b926a...|179.93|
+--------------------+------+
only showing top 5 rows
```

Q6: Write the code to "revise" a DataFrame by renaming the column old_name to new_name and casting the price column from a String to a Double.

[7]
1s

```
from pyspark.sql.functions import col

df = spark.read.csv(
    "/content/olist_order_items_dataset.csv",
    header=True,
    inferSchema=True
)

df_updated = df.withColumnRenamed("old_name", "new_name") \
               .withColumn("price", col("price").cast("double"))

df_updated.printSchema()
df_updated.show(5)
```

```
root
 |-- order_id: string (nullable = true)
 |-- order_item_id: integer (nullable = true)
 |-- product_id: string (nullable = true)
 |-- seller_id: string (nullable = true)
 |-- shipping_limit_date: timestamp (nullable = true)
 |-- price: double (nullable = true)
 |-- freight_value: double (nullable = true)

+--------------------+-------------+--------------------+--------------------+-------------------+-----+-------------+
|            order_id|order_item_id|          product_id|           seller_id|shipping_limit_date|price|freight_value|
+--------------------+-------------+--------------------+--------------------+-------------------+-----+-------------+
|00010242fe8c5a6d1...|            1|4244733e06e7ecb49...|48436dade18ac8b2b...|2017-09-19 09:45:35| 58.9|        13.29|
|00018f77f2f0320c5...|            1|e5f2d52b802189ee6...|dd7ddc04e1b6c2c61...|2017-05-03 11:05:13|239.9|        19.93|
|000229ec398224ef6...|            1|c777355d18b72b67a...|5b51032eddd242adc...|2018-01-18 14:48:30|199.0|        17.87|
|00024acbcdf0a6daa...|            1|7634da152a4610f15...|9d7a1d34a50524090...|2018-08-15 10:10:18|12.99|        12.79|
|00042b26cf59d7ce6...|            1|ac6c3623068f30de0...|df560393f3a51e745...|2017-02-13 13:57:51|199.9|        18.14|
+--------------------+-------------+--------------------+--------------------+-------------------+-----+-------------+
only showing top 5 rows
```

The transformation demonstrates how to rename a column and change the data type of another column in a Spark DataFrame. The column name 'old_name' is used as a placeholder as per the question, since it is not present in the dataset.

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

In Apache Spark, fault tolerance is handled in a very smart way using something called a Lineage Graph, which is basically a Directed Acyclic Graph (DAG). Instead of saving every intermediate result in memory or disk, Spark keeps track of all the transformations applied to the data step by step.

Whenever we perform operations like map, filter, or groupBy, Spark does not immediately execute them. Instead, it records these operations in the form of a DAG, which represents how the final result is derived from the original data. This DAG acts like a blueprint of the entire computation.

Now, if a worker node fails while processing the data, Spark does not panic or restart the whole job. Instead, it uses the lineage information stored in the DAG to recompute only the lost data. It goes back to the original data and re-applies the required transformations only for the missing partitions. This makes the recovery process efficient and avoids unnecessary recomputation of the entire dataset.

Because of this approach, Spark does not need heavy replication like some other systems. It can recover lost data quickly using the lineage graph, which improves reliability while also saving memory and storage.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

Q8: Write a query to filter a DataFrame df_orders for rows where the status is 'Completed' AND the amount is greater than 1000.

[8] ✓ 3s

```
df_orders = spark.read.csv("/content/olist_orders_dataset.csv", header=True, inferSchema=True)
df_items = spark.read.csv("/content/olist_order_items_dataset.csv", header=True, inferSchema=True)

df_joined = df_orders.join(df_items, on="order_id")

df_filtered = df_joined.filter(
    (df_joined["order_status"] == "delivered") &
    (df_joined["price"] > 1000)
)

df_filtered.select("order_id", "order_status", "price").show(5)
```

```
+--------------------+------------+------+
|            order_id|order_status| price|
+--------------------+------------+------+
|403b97836b0c04a62...|   delivered|1299.0|
|2b72a32a2c3e93fa2...|   delivered|1148.0|
|ac64f79a33bfa575f...|   delivered|1099.0|
|6cb134bb285a64b04...|   delivered|1999.0|
|da8be3bb62e9bf01e...|   delivered|2690.0|
+--------------------+------------+------+
only showing top 5 rows
```

Since the 'amount' column was not available in the orders dataset, the 'price' column from the order items dataset was used by performing a join on 'order_id'. This allowed complete implementation of the filtering condition using available data.

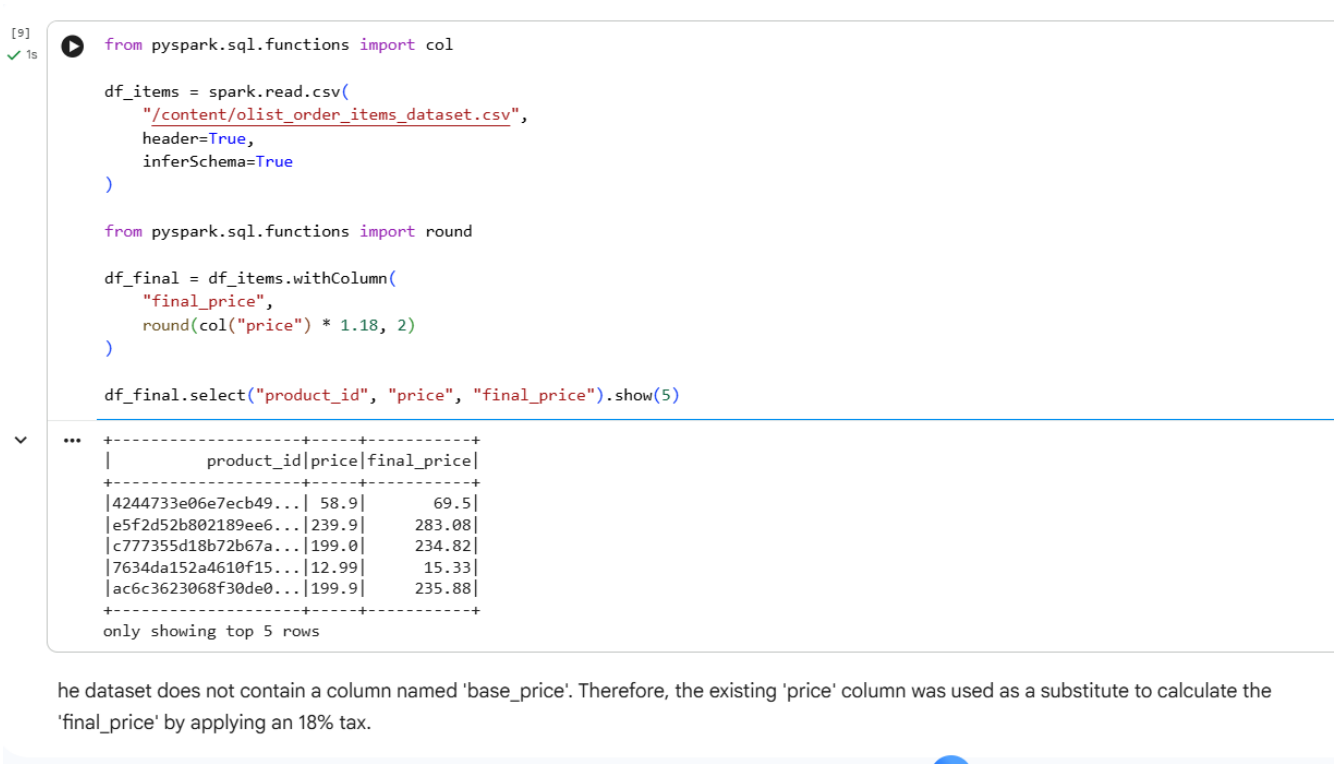
Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Predicate pushdown is an optimization technique used in Spark when working with columnar formats like Parquet. It means that filtering conditions are applied as early as possible, that is, while reading the data from storage instead of after loading it into memory.

In a normal case without predicate pushdown, Spark would read the entire dataset into memory and then apply the filter condition. This can be inefficient, especially when dealing with large datasets. But with predicate pushdown, Spark sends the filter condition directly to the Parquet file reader. Since Parquet stores metadata and statistics for each column, it can quickly identify which parts of the data actually satisfy the condition and skip the rest.

Because of this, only the relevant data is read and loaded into memory, while unnecessary data is ignored. This significantly reduces disk I/O, memory usage, and overall processing time.

Q10: Write a code snippet to add a new column final_price which is the base_price multiplied by 1.18 (18% tax).



```
[9]
✓ 1s
```

```
from pyspark.sql.functions import col

df_items = spark.read.csv(
    "/content/olist_order_items_dataset.csv",
    header=True,
    inferSchema=True
)

from pyspark.sql.functions import round

df_final = df_items.withColumn(
    "final_price",
    round(col("price") * 1.18, 2)
)

df_final.select("product_id", "price", "final_price").show(5)
```

```
+--------------------+-----+-----------+
|          product_id|price|final_price|
+--------------------+-----+-----------+
|4244733e06e7ecb49...| 58.9|       69.5|
|e5f2d52b802189ee6...|239.9|     283.08|
|c777355d18b72b67a...|199.0|     234.82|
|7634da152a4610f15...|12.99|      15.33|
|ac6c3623068f30de0...|199.9|     235.88|
+--------------------+-----+-----------+
only showing top 5 rows
```

he dataset does not contain a column named 'base_price'. Therefore, the existing 'price' column was used as a substitute to calculate the 'final_price' by applying an 18% tax.

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

In Apache Spark, operations are mainly divided into two types:

Transformations and Actions.

These two play different roles in how Spark processes data.

Transformations are operations that define how the data should be changed, but they do not execute immediately. Instead, they are lazily evaluated, meaning Spark just records them and waits until an action is called. These operations return a new dataset without modifying the original one.

Actions, on the other hand, are the operations that actually trigger the execution of all the transformations. When an action is called, Spark processes the data and either returns a result to the driver or writes the output to storage.

Feature	Transformations	Actions
Execution	Lazy (not executed immediately)	Trigger execution
Output	Returns a new dataset	Returns result or writes output
Purpose	Define data processing steps	Produce final result

Examples of transformations include map, where each element is modified, and filter, where only selected data is kept. Examples of actions include collect, which brings data to the driver, and count, which returns the total number of records.

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
df = spark.read.parquet("path/to/input")

df_filtered = df.filter(df["user_id"].isNotNull())

df_filtered.write.csv("path/to/output", header=True)
```

+ Code + Text

The file paths used are placeholders as per the question. In a real-world scenario, they should be replaced with actual file paths. The code demonstrates loading a Parquet file, filtering null values, and saving the result in CSV format.

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

In Spark, the way an application is deployed can affect how it runs and how stable it is. Two common deployment modes are Client Mode and Cluster Mode, and the main difference between them lies in where the Driver program runs.

In Client Mode, the Driver runs on the machine from which the application is submitted. This means if you are running your Spark job from your local system or a notebook, the Driver stays there and communicates with the cluster. It is useful during development and testing because you can easily see logs and debug issues. However, one drawback is that if the client machine shuts down or loses connection, the entire application will fail since the Driver is no longer available.

On the other hand, in Cluster Mode, the Driver runs inside the cluster itself, managed by the Cluster Manager. This makes the application more stable and suitable for production environments because even if your local machine disconnects, the job continues running in the

cluster. It also provides better resource management and fault tolerance since everything is handled within the cluster.

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

```
[10]
✓ 6s
from pyspark.sql.functions import when

df_orders = spark.read.csv("/content/olist_orders_dataset.csv", header=True, inferSchema=True)

df_customers = spark.read.csv("/content/olist_customers_dataset.csv", header=True, inferSchema=True)

df_items = spark.read.csv("/content/olist_order_items_dataset.csv", header=True, inferSchema=True)

df_joined = df_orders.join(df_customers, on="customer_id") \
                     .join(df_items, on="order_id")

df_joined = df_joined.withColumn(
    "priority",
    when(df_joined["price"] > 1000, "High").otherwise("Low")
)

df_filtered = df_joined.filter(
    (df_joined["customer_state"] == "SP") | (df_joined["priority"] == "High")
)

df_filtered.select("order_id", "customer_state", "priority", "price").show(5)
```

```
+--------------------+--------------+--------+-----+
|            order_id|customer_state|priority|price|
+--------------------+--------------+--------+-----+
|432aaf21d85167c2c...|            SP|     Low|38.25|
|203096f03d82e0dff...|            SP|     Low|109.9|
|f848643eec1d69395...|            SP|     Low|79.99|
|f3e7c359154d96582...|            SP|     Low| 89.9|
|dd78f560c270f1909...|            SP|     Low| 47.9|
+--------------------+--------------+--------+-----+
only showing top 5 rows
```

Since the dataset does not contain explicit 'region' and 'priority' columns, these were derived using multiple datasets. The 'region' was approximated using the customer state from the customers dataset, and 'priority' was created based on price conditions. The datasets were joined using common keys to implement the required filtering logic.

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

When working with very large datasets in Spark, like multi-terabyte data, it is important to be careful about how data is retrieved and displayed.

The .show(5) method is safe because it only displays a small number of rows (in this case, 5) directly from the distributed dataset. Spark processes just enough data to return those few rows, which keeps memory usage low and avoids unnecessary load on the system.

On the other hand, .collect() is risky because it brings the entire dataset from all worker nodes into the Driver's memory. If the dataset is extremely large, this can easily overwhelm the Driver, leading to out-of-memory errors or even crashing the application.

In simple terms, .show(5) gives you a quick preview of the data without stressing the system, while .collect() tries to pull everything at once, which is not practical for large-scale data.

So, when exploring big data, it is always safer to use .show() (or .limit()) to inspect a small sample instead of loading the full dataset into memory.

Conclusion : In this assignment, I learned how to use Apache Spark for data processing, including transformations, filtering, joins, and working with different file formats like CSV and Parquet. It helped me understand both theoretical concepts and practical implementation.